

3-3 · DuckDB를 이용한 로컬 대용량 데이터 처리 · 실습 1.5H

대용량 변환 실습

COPY · 크기·속도 실측

CSV 를 Parquet 로 — 크기와 속도를 직접 쟀다

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① COPY 구문으로 events.csv 를 zstd 압축 Parquet 로 변환한다
- ② 파일 크기 배율과 같은 질의의 속도 배율을 3회 평균으로 잰다
- ③ 한 열만 읽기 vs 전부 읽기로 '필요한 열만 읽는다'를 검증한다
- ④ 압축 3종을 비교하고 PARTITION_BY 로 값별 폴더를 만든다

잰 수치는 실측 기록 표에 적어 정리·회고에서 옆 사람과 비교합니다

일곱 단계 — 변환하고, 재고, 이유를 확인하고, 기록

앞 시간의 events.csv 와 bench 함수(3회 평균)를 그대로 씁니다

1. COPY 변환

events.parquet
zstd

2.3. 크기·속도

getsize
bench 3회 평균

4. 왜 빠르냐

한 열만 vs
전부 읽기

5.6. 압축·파티션

3종 비교
PARTITION_BY

7. 실측 기록

표에 배울
결론 한 문장

1~3단계

핵심 명령 — 변환 · 크기 · 속도

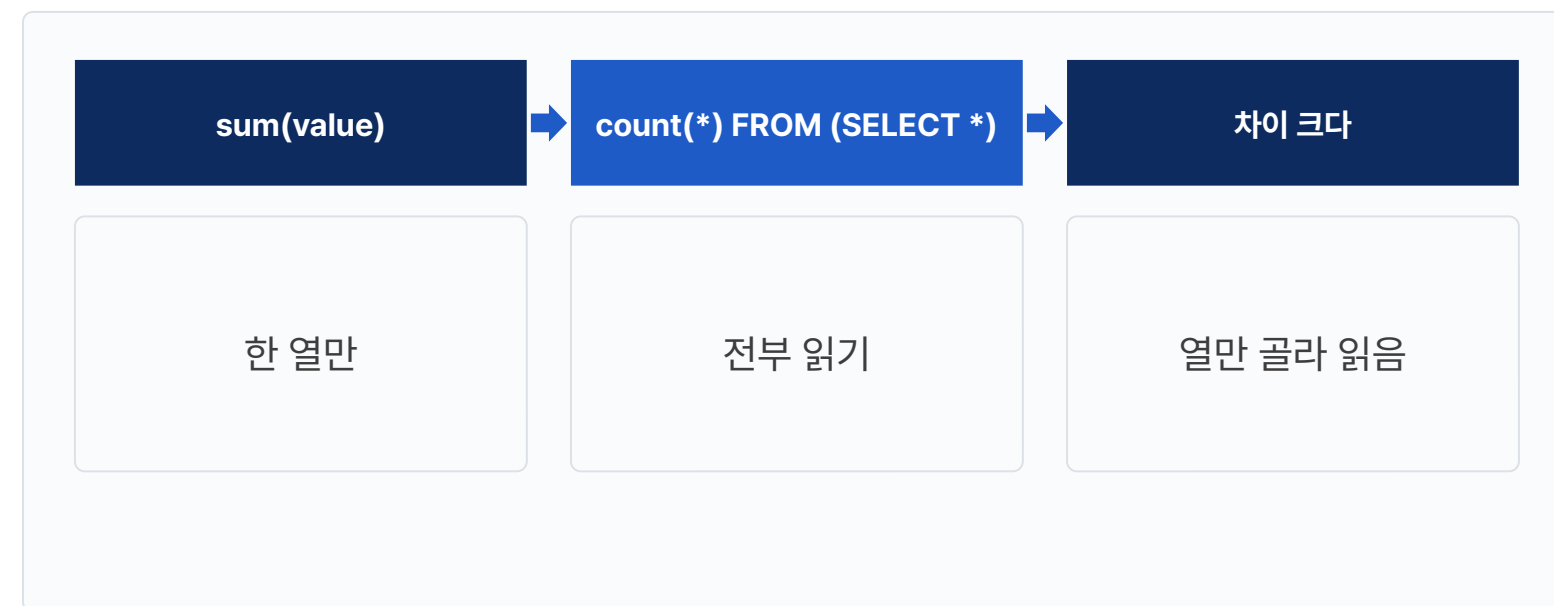
하려는 일	코드	확인
Parquet 변환	<code>COPY (SELECT * FROM 'events.csv') TO 'events.parquet' (FORMAT parquet, COMPRESSION zstd)</code>	「변환 완료」· 파일 생김
크기 비교	<code>ls -lh events.csv events.parquet · os.path.getsize()</code>	MB 와 배율
속도 재기	<code>bench(sql, rounds=3) — duckdb.sql(sql).fetchall()</code>	초 단위 평균
같은 질의 둘	<code>SELECT event_type, sum(value) ... GROUP BY event_type</code>	CSV 와 Parquet 각각

한 번만 재면 우연이 섞인다 — bench 는 perf_counter 로 3회 재서 평균을 돌려준다

왜 빠르냐 — 한 열만 읽기 vs 전부 읽기

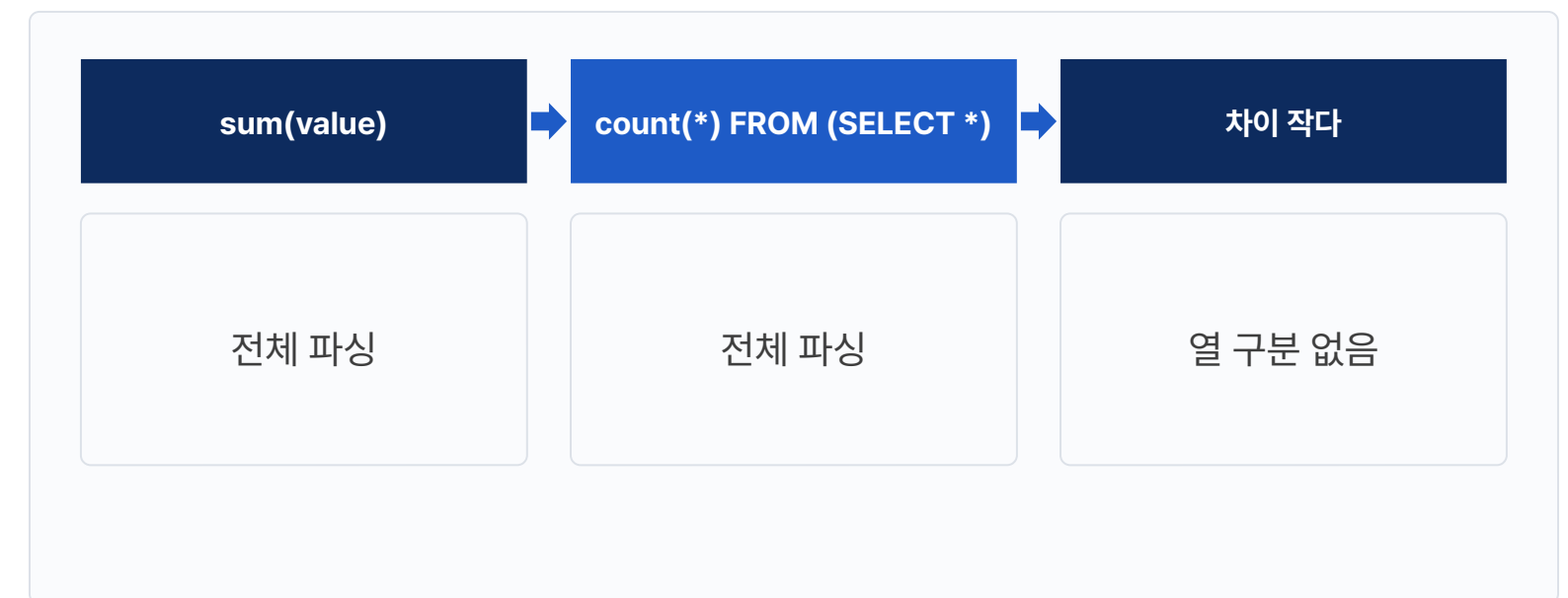
Parquet

한 열만 쓸 때가 눈에 띄게 빠르다



CSV

어느 쪽이든 전체를 파싱해야 해 차이가 작다



VS

압축 3종 비교 — for comp in zstd · snappy · gzip

COMPRESSION	출력 파일	확인 포인트
zstd	events_zstd.parquet	대체로 가장 작다 — 기본
snappy	events_snappy.parquet	속도 순위는 환경마다 다르다
gzip	events_gzip.parquet	본인 수치를 기록

세 줄이 「이름 · MB · 초」로 찍힌다 — 크기와 속도를 실측 기록 표에 옮긴다

SELECT * 습관이 대용량에서 그대로 비용이 되는 이유 — 열 50개 중 3개만 필요한데 * 를 쓰면 47개를 헛읽는다

파티션 COPY 를 두 번 — 둘째가 왜 실패하는지

PARTITION_BY (event_type) 로 저장하고, OVERWRITE_OR_IGNORE 를 뺀 채 한 번 더 실행해 봅니다

PARTITION_BY	find · du -sh	옵션 빼고 재실행	실패	옵션 되살려 성공
TO 'events_parted' event_type 별	event_type=view/ 폴더가 값별로	덮어쓰기 옵션 없이 한 번 더	Directory is not empty	옵션 되살리기 또는 폴더 삭제

폴더 이름이 곧 조건 — buy 만 읽을 때 다른 폴더는 안 연다

1 FROM 'events_parted/*/*.parquet' WHERE event_type = 'buy'
파티션된 쪽 — view.cart 폴더는 아예 열지 않는다

2 FROM 'events.parquet' WHERE event_type = 'buy'
단일 파일 — 전체에서 골라낸다

3 빠르거나 비슷하게 나온다
지금은 데이터가 작아 차이가 작을 수 있다 — 수억 행에서는 결정적

4주차 예고

Iceberg 가
이 원리를 정교하게

실측 기록 표 — 3-5 설계서와 주차 산출물의 근거

항목	CSV	Parquet(zstd)	배율
파일 크기	MB	MB	배
집계 질의	초	초	배
한 열만 읽기	초	초	배
압축 비교	zstd MB	snappy MB	gzip MB

결론 한 문장 — 왜 중간 산출물을 Parquet 로 두는가

수치는 환경마다 다르다 — 옆 사람과 배율이 달라도 이상하지 않다. 중요한 것은 방향: Parquet 가 작고 빠르다, 열을 적게 읽을수록 유리하다

이 1.5시간이 끝날 때 확인할 것

- 1 events.parquet 가 생겼고 CSV 보다 확연히 작다
- 2 같은 집계 질의가 Parquet 에서 더 빠르다(3회 평균)
- 3 Parquet 는 한 열만 읽을 때 눈에 띄게 빠르고, CSV 는 차이가 작다
- 4 압축 3종 중 zstd 가 대체로 가장 작다
- 5 Directory is not empty 를 봤고 OVERWRITE_OR_IGNORE 로 해결했다
- 6 실측 기록 표에 배율과 결론 한 문장을 적었다

3-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

DuckDB 문법을 한 장으로 정리하고, 실측 수치를 옆 사람과 비교합니다

✓ DuckDB 요약 표 — 하려는 일과 코드

✓ 실측 수치 공유 — SSD·HDD 차이

✓ 자주 겪는 오류 네 가지

✓ 3-4 DuckDB 데이터 분석 예고

가져갈 것

실측 기록 표(배율 3개 · 압축 3종)